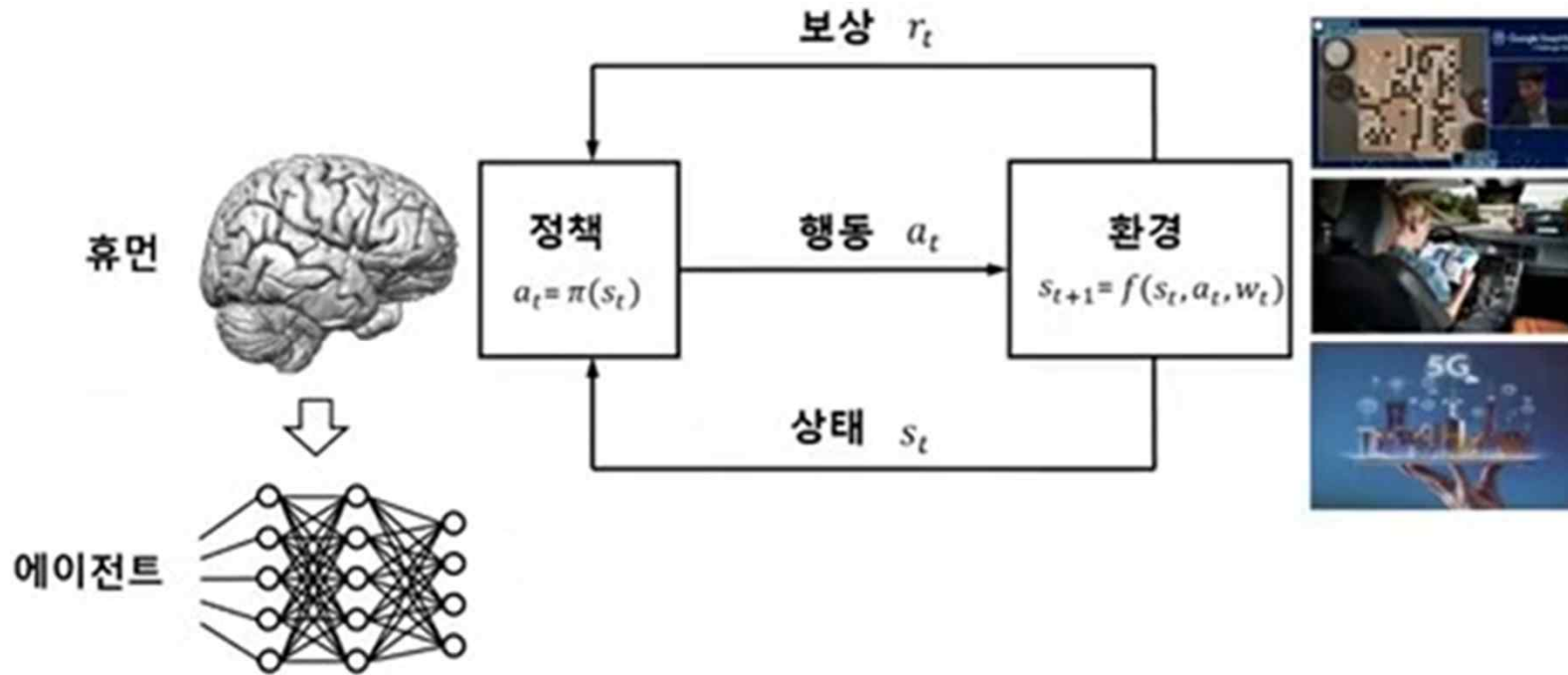


강화학습 기초

✓ 불확실하고 복잡한 세상의 연속 의사결정

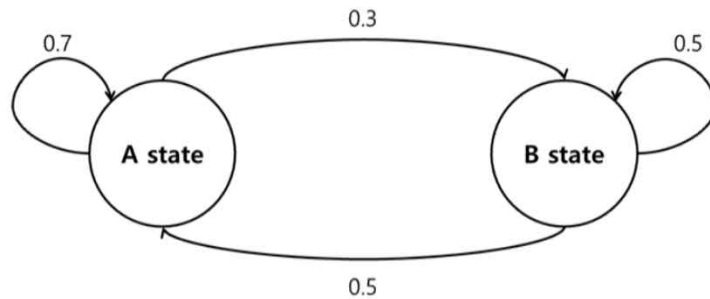


✓ 최적의 의사결정을 하려고 그러면 내가 어떤 의사결정을 해서 행동을 했을 때 세상이 어떻게 반응할지를 미리 알 수 있어야 최적의 전략을 짤 수 있음

- 세상의 반응에 대하여 모르기 때문에(불확실성) 최적의 의사결정이 어려움
- 문제가 너무나 커서 계산이 어려움

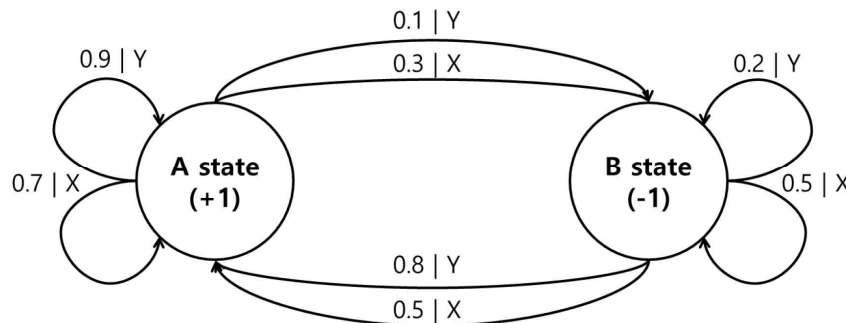
✓ 1900년대 마르코프 의사결정

- Discrete state space : $S = \{A, B\}$
- State transition probability : $P(S'|S) = \{P_{AA} = 0.7, P_{AB} = 0.3, P_{BA} = 0.5, P_{BB} = 0.5\}$
- Purpose : Finding a steady state distribution



Andrey Markov
1856 - 1922

- Discrete state space : $S = \{A, B\}$
- Discrete action space : $A = \{X, Y\}$
- (Action conditional) State transition probability : $P(S'|S, A) = \{ \dots \}$
- Reward function : $R(S'=A) = +1, R(S'=B) = -1$
- Purpose : Finding an optimal policy (maximizes the expected sum of future reward)



행동가치 Q



행동 $a=2$ 의 가치

$$= 0.5 \times (5 + 20) + 0.2 \times (2 + 25) + 0.3 \times (11 + 17) = 26.3$$

행동가치 (1952년)



Richard Bellman (1920-1984)

행동가치함수 $Q(s, a)$

	$a = 1$	$a = 2$
$s = 0$	15.2	26.3
$s = 1$	8.0	4.1
$s = 2$	25.1	36.0

"Q-테이블"

행동가치 Q 학습

행동가치 학습 (1989년)



Christopher Watkins

영국 캠브리지 대학 크리스토 워킨스라

✓ **경험 데이터** (s, a, r, s')

$$(s_1, a_1, r_1, s'_1) \rightarrow (s_2, a_2, r_2, s'_2) \rightarrow (s_3, a_3, r_3, s'_3) \rightarrow \dots$$

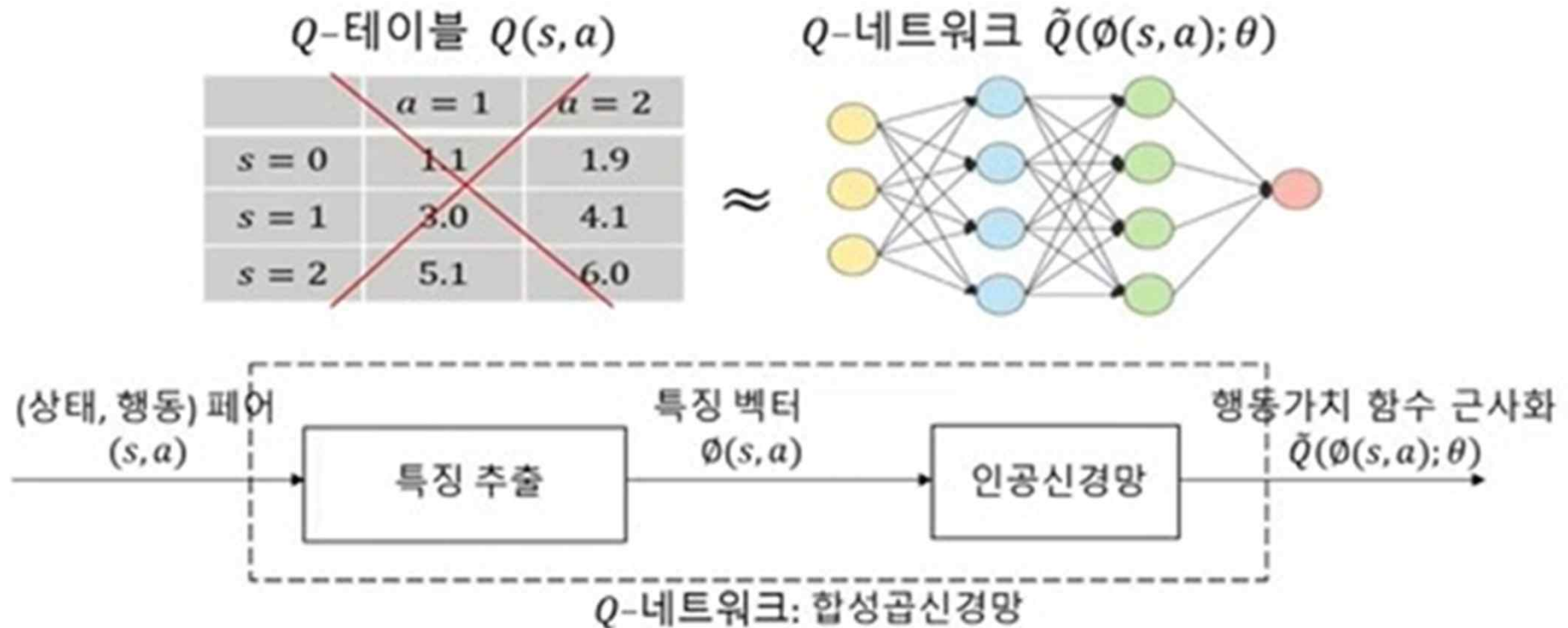
✓ **경험 데이터 수집을 통해 불확실성의 저주 문제 해결 → “Q-학습”**

✓ **경험 데이터 수집 방법**

- 실제 경험
- 시뮬레이션/디지털트윈을 통한 가상 경험
- 생성 모델을 통한 위조 경험

행동가치 Q 근사화

- ✓ 미해결 과제인 복잡도의 저주 문제 해결 방법 → “Q-Network”



DQN : 딥 Q-네트워크

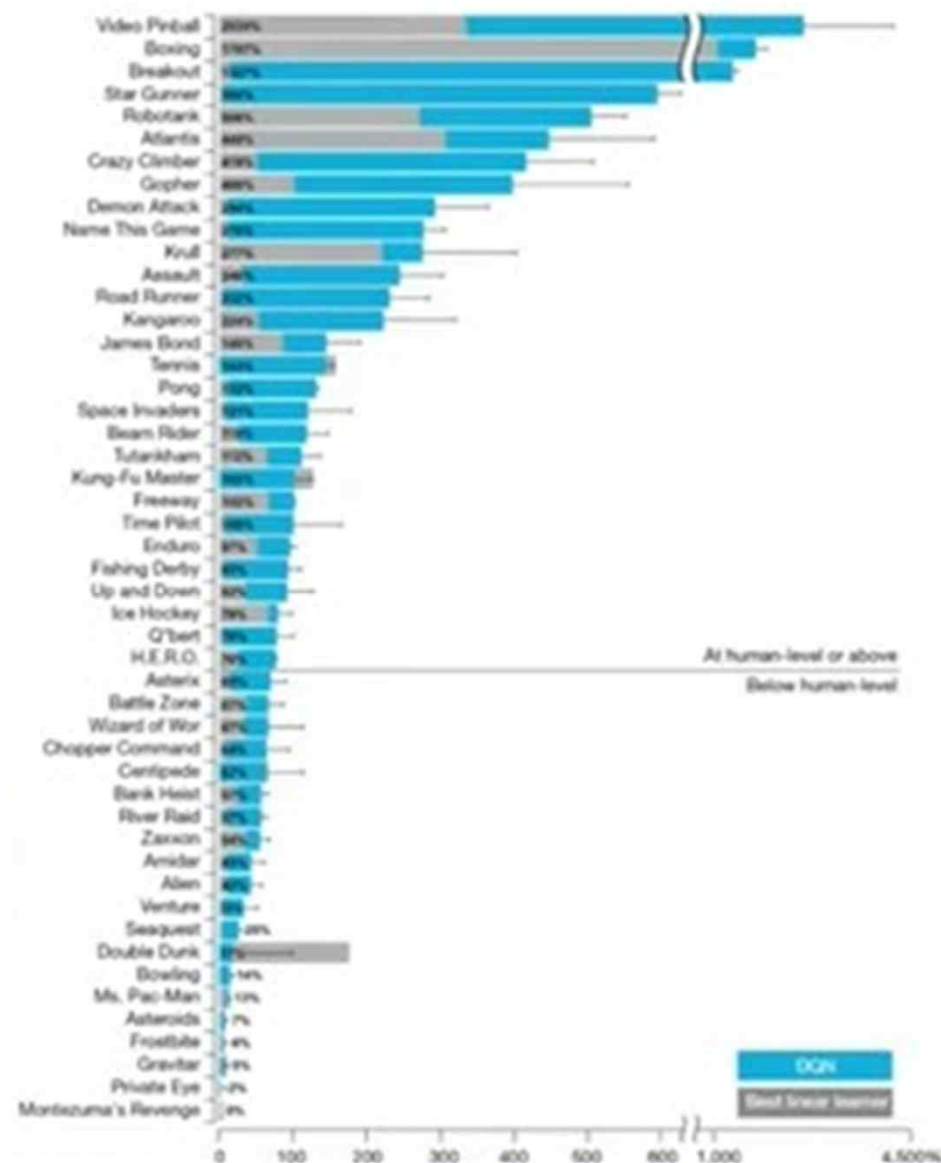
Nature (2013)



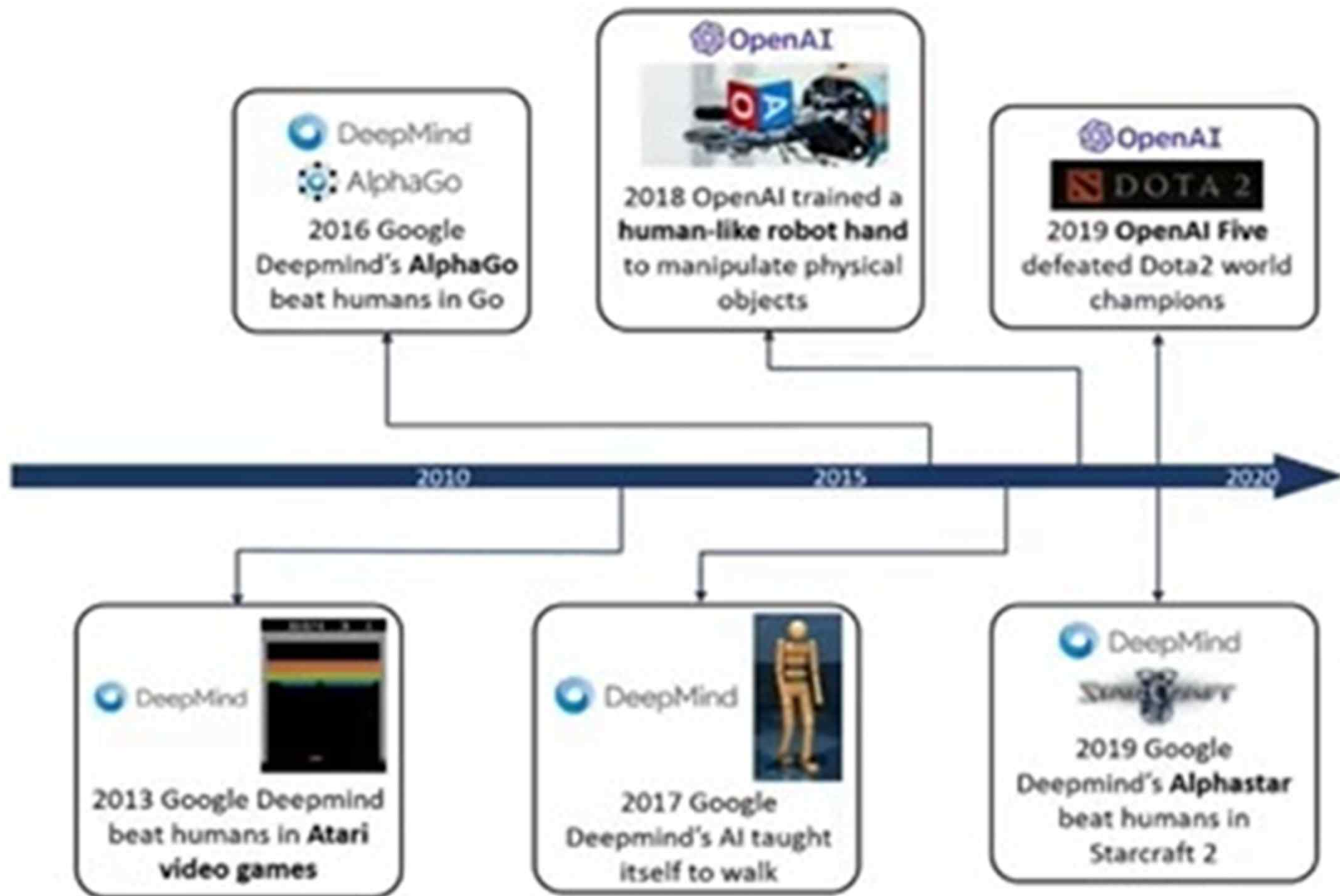
브레이크아웃:
인간 전문가 보다
13배 높은 점수



몬테주마의 복수:
학습 실패



심층강화학습 발전



확률 부터 강화학습의 관계

확률 :

표본공간(Ω) > 확률변수(X) > 확률분포함수($P(X)$) > 확률과정함수($P(X_t)$) > 마르코프과정 > 마르코프 체인 > 마르코프의사결정

순차의사결정 방식(MDP : S A P R r)

$s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow e$

벨만방정식

Reward > return(G) > Value function(V) > Q function(Q) > Policy(π) > E-greedy

Planning :

Dynamic Program
Value Iteration/Policy Iteration

Sampling :

몬테카를로 제어(MC), 시간차 예측(TD)
SARSA, Q-learning

Planning

Dynamic program
Value Iteration/Policy Iteration

Learning

MC, TD, e-greedy,
SARSA, Q-learning

순차의사결정

MDP : S A P R r
벨만 방정식 : G $V(s)/Q(s)$ $\pi(s)$

확률

Ω , X , $P(X)$, X_t , $P(X_t)$,
마르코프 과정, 마르코프 체인, 마르코프 의사 결정

MDP

(Markov Decision Proccss)

순차적 행동 결정 문제를 정의한다.
상태, 행동, 보상 함수, 상태 변환 확률, 감가율

A Markov decision process (MDP) is a Markov reward process with decisions. It is an *environment* in which all states are Markov.

Definition

A *Markov Decision Process* is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P}$ is a state transition probability matrix,
 $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$
- $\mathcal{R}$ is a reward function, $\mathcal{R}_s^a = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$
- γ is a discount factor $\gamma \in [0, 1]$.

- ✓ 상태(S_t) : 현재 상황을 나타낸다.
 - 환경이 에이전트에 제공해 준다.
 - 실물에서는 센서값을 상태로 봄
- ✓ 확률과정으로 나타냄
 - 어떤 t 에서 상태 S_t 는 정해져 있지 않는다.
 - 때에 따라서 $t=1$ 일 때 $S_t=1$ 일 수도 있고 $S_t=4$ 일 수도 있다.
- ✓ 시간 t 에서 확률변수 상태
 - $S_t=s$

Frozen Lake에는 16개의 상태가 있다.

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	F_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

- ✓ 에이전트가 상태 S_t 에서 할 수 있는 가능한 행동의 집합 A
- ✓ $A_t = a$
 - 시간 t 에서의 행동 a
- ✓ $A = \{up, down, left, right\}$
 - 그리드월드에서 에이전트가 할 수 있는 행동의 집합
- ✓ 상태 변환 확률
 - 바람과 같은 예상치 못한 요소에 의해서 목적 상태에 도달하지 못하는 것을 고려 함

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

보상함수(Reward function)

S A R P r

✓ $R_s^a = E[R_{t+1}|S_t=s, A_t=a]$

✓ E 기대 함수 expectation value

□ 일종의 평균

✓ 상태 s에서 행동 a를 했을 경우에 받을 것이라 예상되는 숫자

✓ 보상함수를 기대 값으로 표현하는 것은

□ 환경에 따라서 같은 상태에서 같은 행동을 취하더라도 다른 보상을 줄 수도 있어서 이를 고려하기 위해

✓ Reward function

✓ $R_s^a = E[R_{t+1}|S_t=s, A_t=a]$

✓ | 조건문

□ | 뒤에 있는 것이 현재의 조건

✓ t+1

□ 행동하는 것은 t에서인데 보상을 받는 것은 t+1 이라는 것이다.

□ 보상을 agent가 알고 있는 것이 아니고 환경이 알려주기 때문이다.

□ 상태 S에서 행동 a를 하면 환경은 에이전트가 가게 되는 다음 상태 s' 과 에이전트가 받을 보상을 에이전트에게 알려준다.

□ 환경이 에이전트에게 알려주는 것이 t+1인 시점이다.

□ 따라서 에이전트가 받는 보상을 R_{t+1} 이라고 표현한다.

S, F : 0
H : -1
G : 1

S ₀	F ₀	F ₀	F ₀
F ₀	H ₋₁	F ₀	H ₋₁
F ₀	F ₀	F ₀	H ₋₁
H ₋₁	F ₀	F ₀	G ₁

상태 전이 확률(State transition probability)

SARPr

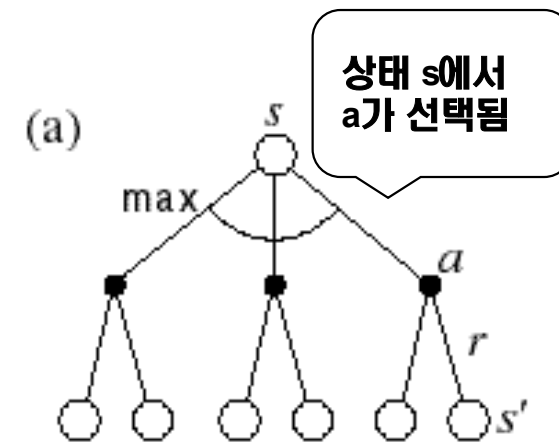
✓ $P_{ss'}^a = P[S_{t+1}=s' | S_t=s, A_t=a]$

- 에이전트가 상태 s 에서 행동 a 를 해서 s' 으로 가게 되는 확률
- 이 값은 보상과 마찬가지로 에이전트가 알지 못하는 값으로서 에이전트가 아닌 환경의 일부

✓ 상태 변환 확률은 환경 모델이라고 한다.

- 환경은 에이전트가 행동을 취하면 상태 변환 확률을 통해 다음에 에이전트가 갈 상태를 알려 준다.

S	F	F	F
F	H	F	H
F	???	F	H
H	F	F	G



a 가 선택이 된 이후에 선택되어질 상태가 여러 개 있으며 이를 확률로 표시한다.

✓ 시간에 따라서 감가하는 비율을 감가율이라고 한다.

✓ 감가율의 정의

□ $\gamma \in [0, 1]$

□ 감가율 γ 0과 1 사이의 값이다.

✓ 감가율을 고려한 미래 보상의 현재 가치

□ $\gamma^{k-1} R_{t+k}$

□ 현재부터 시간이 k 만큼 지났기 때문에 미래에 받을 보상 R_{t+k} 는 γ^{k-1} 만큼 감가된다.

□ 더 먼 미래에 받을 수록 에이전트가 받는 보상의 크기는 줄어든 것이다.

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + R_{t+4} + R_{t+5} + \dots$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \gamma^4 R_{t+5} + \dots$$

S ₁	F ₁	F ₁	F
F	H	F ₁	H
F	F	F ₁	H
H	F	F ₁	G

S ₉ ^{0.5904}	F ^{0.6561}	F _{0.729}	F
F	H	F _{0.81}	H
F	F	F _{0.9}	H
H	F	F ₁	G

벨만방정식 - 가치함수, 큐함수, 정책

순차적 행동 결정 문제를 풀다

Planning : Exhaustive Search, Dynamic Programming

Learning : Monte Carlo, TD, SARSA, Q-learning

가치함수, 큐함수, 정책 관련 내용 정리



R

G

V/Q

π

✓ 시간 t 에서 행동을 통하여 받을 보상의 합(G)

□ $R_{t+1} + R_{t+2} + R_{t+3} + R_{t+4} + R_{t+5} + \dots$

✓ 감가율을 적용한 보상의 합(G)

□ $G_t = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \gamma^4 R_5 + \dots$

□ $= \sum_{k=1}^{\infty} \gamma^{k-1} R_{t+k}$

✓ 가치 함수

□ $v(s) = E[G_t | S_t = s]$

✓ 큐함수

□ $q(s) = E[G_t | S_t = s, A_t = a]$

✓ 정책

□ $\pi(s) = P[A_t = a | S_t = s]$

□ $\pi'(s) = \operatorname{argmax}_a q_{\pi}(s, a)$

가치 함수, 큐함수

✓ 가치 함수

- 현재 상태에서 미래에 받을 보상의 합에 대한 기대값
 - 기대값을 취하는 이유는 현재상태에는 여러 액션이 있기 때문에 각 액션에 따른 미래에 대하여 받을 보상의 합에 대한 평균을 말하고 있다.

✓ 큐 함수

- 현재 상태에서 행동을 취한 경우 미래에 받을 보상의 합에 대한 기대값
 - 기대값을 취하는 이유는 액션에 따라 상태전이 확률이 있기 때문에 각 각 상태에 따른 미래에 대하여 받을 보상의 합에 대한 평균을 말하고 있다.
- 행동을 결정할 때도 확률이 있고 결정된 행동에서 상태로 갈 때도 확률이 있다.

V_tbl[16]로 할당됨

S _{V(0)}	F _{V(1)}	F _{V(2)}	F _{V(3)}
F _{V(4)}	H _{V(5)}	F _{V(6)}	H _{V(7)}
F _{V(8)}	F _{V(9)}	F _{V(10)}	H _{V(11)}
H _{V(12)}	F _{V(13)}	F _{V(14)}	G _{V(15)}

Q_tbl[16][4]로 할당

Q(0,0) S	Q(0,1) F	Q(0,2) F	Q(0,3) F
Q(1,0) F	Q(1,1) F	Q(1,2) F	Q(1,3) F
Q(2,0) F	Q(2,1) F	Q(2,2) F	Q(2,3) F
Q(3,0) F	Q(3,1) F	Q(3,2) F	Q(3,3) F
Q(4,0) F	Q(4,1) H	Q(4,2) F	Q(4,3) H
Q(5,0) H	Q(5,1) H	Q(5,2) F	Q(5,3) H
Q(6,0) F	Q(6,1) F	Q(6,2) F	Q(6,3) F
Q(7,0) H	Q(7,1) H	Q(7,2) F	Q(7,3) H
Q(8,0) F	Q(8,1) F	Q(8,2) F	Q(8,3) F
Q(9,0) F	Q(9,1) F	Q(9,2) F	Q(9,3) F
Q(10,0) F	Q(10,1) F	Q(10,2) F	Q(10,3) F
Q(11,0) H	Q(11,1) H	Q(11,2) F	Q(11,3) H
Q(12,0) H	Q(12,1) F	Q(12,2) F	Q(12,3) F
Q(13,0) F	Q(13,1) F	Q(13,2) F	Q(13,3) F
Q(14,0) F	Q(14,1) F	Q(14,2) F	Q(14,3) F
Q(15,0) G	Q(15,1) G	Q(15,2) F	Q(15,3) G

Value Function VS Q- Function

Value function : State만 고려

Definition

The state-value function $v_{\pi}(s)$ of an MDP is the expected return starting from state s , and then following policy π

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid S_t = s]$$

Q-function : State와 Action을 같이 고려

Definition

The action-value function $q_{\pi}(s, a)$ is the expected return starting from state s , taking action a , and then following policy π

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a]$$

Value function은 각 action에 확률과 Q-function값의 합

$$v_{\pi}(s) = \sum_{a \in A} \pi(a \mid s) q_{\pi}(s, a)$$

정책 Policy

✓ 정책을 모든 상태에서 에이전트가 할 행동

- 상태가 입력으로 들어오면 행동을 출력으로 내보내는 일종의 함수
- 정책은 각 상태에서 단 하나의 행동만을 나타낼 수도 있고 확률적으로 $a1 = 10\%$, $a2 = 90\%$

✓ 최적 정책

- 에이전트가 강화 학습을 통하여 학습해야 하는 것

$$\pi(a | s) = P[A_t = a | S_t = s]$$

$$\pi'(s) = \underset{a}{\operatorname{argmax}} q_{\pi}(s, a)$$

벨만 방정식

벨만방정식이라고 쓰고 다이나믹 프로그래밍이라 부른다.

Planning – model base

가치함수의 벨만 방정식

- ✓ 현재 상태에서 미래에 받을 보상의 합에 대한 기대값

$$V(s) = E[G_t \mid S_t = s]$$

$$E[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

$$E[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s]$$

$$E[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

$$E[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s]$$

$$V^\pi(s) = \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')]$$

V_tbl[16]로 할당됨

S G_t	F G_{t+1} R_{t+1}	F G_{t+2} R_{t+2}	F $V(3)$
$V(4)$ F	H $V(5)$	F G_{t+3} R_{t+3}	H $V(7)$
$V(8)$ F	F $V(9)$	F G_{t+4} R_{t+4}	H $V(11)$
$V(12)$ H	F $V(13)$	F G_{t+5} R_{t+5}	G $V(15)$

$$\begin{aligned}
 G_t &= r_0 R_{t+1} + r_1 R_{t+2} + r_2 R_{t+3} + r_3 R_{t+4} + r_4 R_{t+5} = R_{t+1} + r G_{t+1} \\
 G_{t+1} &= r_0 R_{t+2} + r_1 R_{t+3} + r_2 R_{t+4} + r_3 R_{t+5} = R_{t+2} + r G_{t+2} \\
 G_{t+2} &= r_0 R_{t+3} + r_1 R_{t+4} + r_2 R_{t+5} = R_{t+3} + r G_{t+3} \\
 G_{t+3} &= r_0 R_{t+4} + r_1 R_{t+5} = R_{t+4} + r G_{t+4} \\
 G_{t+4} &= r_0 R_{t+5} = R_{t+5}
 \end{aligned}$$

큐함수의 벨만방정식

$$q(s) = E[G_t \mid S_t = s, A_t = a]$$

$$E[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s, A_t = a]$$

$$E[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s, A_t = a]$$

$$E[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a]$$

$$E[R_{t+1} + \gamma q(S_{t+1}) \mid S_t = s, A_t = a]$$

$$q^\pi(s) = \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma q^\pi(s')]$$

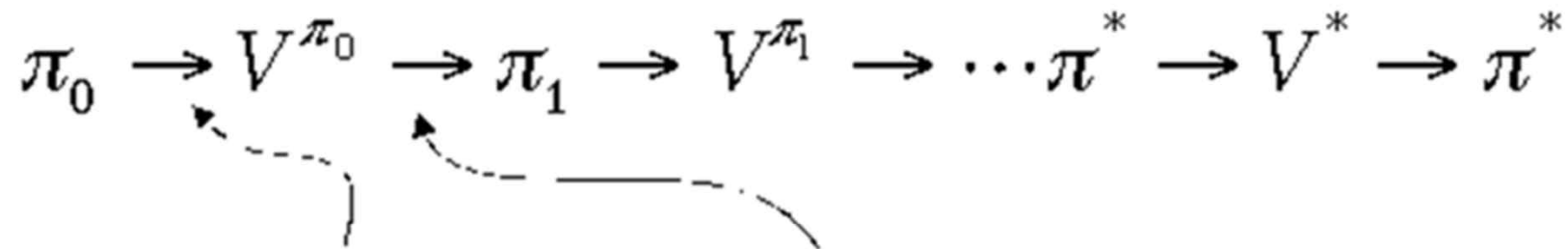
Policy Iteration

- ✓ Policy iteration consists of two simultaneous, interacting processes.
- ✓ Policy evaluation (Iterative formulation):
 - First, make the value function more precise with the current policy.
- ✓ Policy improvement (Greedy selection):
 - Second, make greedy policy greedy with respect to the current value function.
- ✓ Policy Iteration = Policy Evaluation + Policy Improvement

$$\begin{aligned}v_{\pi}(s) &\doteq \mathbb{E}_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s] \\&= \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s] \\&= \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_{\pi}(s')],\end{aligned}$$

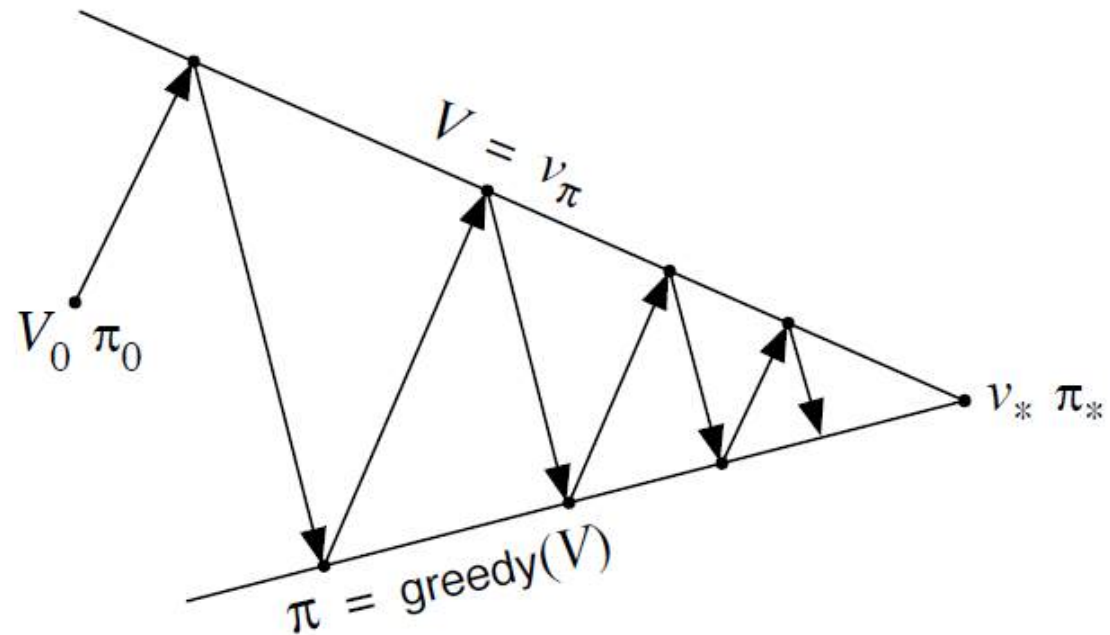
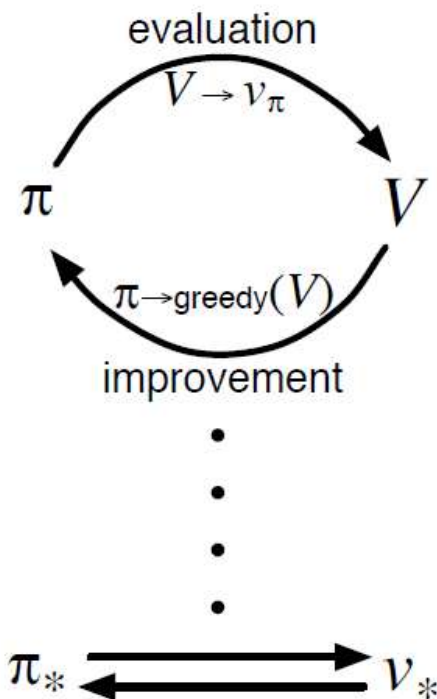
$$\begin{aligned}\pi'(s) &\doteq \operatorname{argmax}_a q_{\pi}(s, a) \\&= \operatorname{argmax}_a \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t = a] \\&= \operatorname{argmax}_a \sum_{s', r} p(s', r|s, a) [r + \gamma v_{\pi}(s')],\end{aligned}$$

Policy Iteration



policy evaluation

policy improvement
“greedification”



Monte Carlo부터 Q-learning까지

Sampling이라고 쓰고 learning 이라고 읽는다.

Model free

여기서 부터 강화학습

Monte Carlo

$$\begin{aligned}V_{n+1} &= \frac{1}{n} \sum_{i=1}^n G_i = \frac{1}{n} (G_n + \sum_{i=1}^{n-1} G_i) \\V_{n+1} &= \frac{1}{n} (G_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} G_i) \\V_{n+1} &= \frac{1}{n} (G_n + (n-1)V_n) \\V_{n+1} &= \frac{1}{n} (G_n + nV_n - V_n) \\V_{n+1} &= V_n + \frac{1}{n} (G_n - V_n) \\V_{n+1} &= V_n + \alpha (G_n - V_n)\end{aligned}$$



$$\sum_{i=1}^n G_i$$

$$\checkmark \quad V_{n+1} = \frac{1}{n} (G_1 + G_2 + G_3 + \cdots + G_{n-2} + G_{n-1} + G_n)$$

$$\sum_{i=1}^{n-1} G_i$$

$$(n-1) \frac{1}{(n-1)} \sum_{i=1}^{n-1} G_i$$

$$V_n = \frac{1}{(n-1)} \sum_{i=1}^{n-1} G_i$$

$$(n-1)V_n$$

$$= \frac{1}{n} ((n-1)V_n + G_n) = \frac{1}{n} (nV_n - V_n + G_n) = \frac{1}{n} (nV_n + G_n - V_n) = V_n + \frac{1}{n} (G_n - V_n)$$

$$= V_n + \alpha (G_n - V_n)$$

Monte Carlo 예측

- ✓ 다이내믹 프로그래밍에서 강화학습으로 넘어가는 가장 기본적인 아이디어
- ✓ 기대값을 샘플링을 통한 평균으로 대체하는 기법
- ✓ 에피소드를 하나 진행하고 에피소드 동안 지나온 상태의 반환값을 구함
- ✓ 이 반환값은 하나의 샘플이 되어서 각 상태의 가치함수를 업데이트 함

Initialize:

$\pi \leftarrow$ policy to be evaluated

$V \leftarrow$ an arbitrary state-value function

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Repeat forever:

Generate an episode using π

For each state s appearing in the episode:

$G \leftarrow$ return following the first occurrence of s

Append G to $Returns(s)$

$V(s) \leftarrow \text{average}(Returns(s))$

Temporal-Difference Learning 시간차 예측

- ✓ 타임스텝마다 가치함수를 업데이트함
- ✓ 벨만기대 방정식을 이용하여 가치함수를 업데이트 한다.

```
Input: the policy  $\pi$  to be evaluated
Initialize  $V(s)$  arbitrarily (e.g.,  $V(s) = 0, \forall s \in \mathcal{S}^+$ )
Repeat (for each episode):
  Initialize  $S$ 
  Repeat (for each step of episode):
     $A \leftarrow$  action given by  $\pi$  for  $S$ 
    Take action  $A$ , observe  $R, S'$ 
     $V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$ 
     $S \leftarrow S'$ 
  until  $S$  is terminal
```

E-greedy

- ✓ 일정한 확률로 랜덤하게 행동 선택

탐욕정책 $\pi'(s) = \operatorname{argmax}_a q_\pi(s, a)$

ε -탐욕정책 $\pi(s) = \begin{cases} a^* = \operatorname{argmax}_a q(s, a), & 1 - \varepsilon \\ a \neq a^*, & \varepsilon \end{cases}$

SARSA 제어 – 시간차 제어

- ✓ 행동을 선택할 때 ϵ -탐욕 정책 사용 -> 가치함수를 사용하지 않고 큐함수 사용
 - 가치함수는 환경모델을 알아야함
- ✓ 하나의 샘플로 (s, a, r, s', a') 필요함
- ✓ 온폴리시 강화학습

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$

Repeat (for each episode):

Initialize S

Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Repeat (for each step of episode):

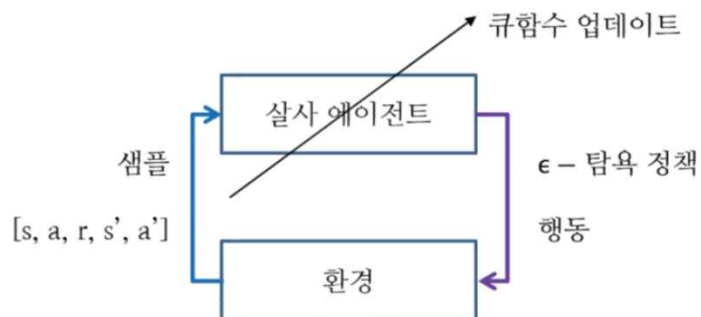
Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

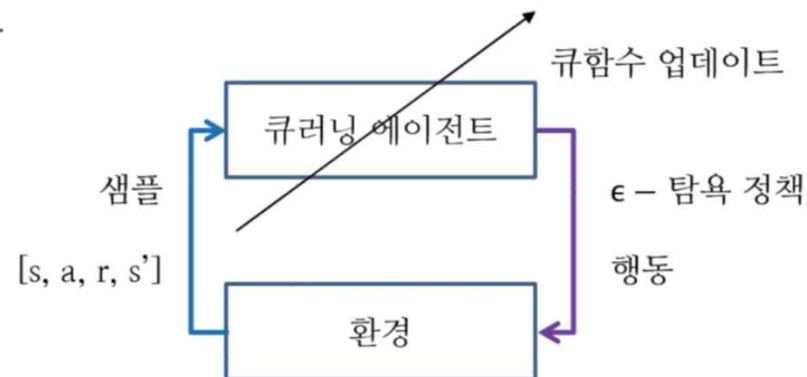
until S is terminal



Q-learning

- ✓ 오프폴리시 강화학습
- ✓ 행동을 선택할 때 ϵ -탐욕 정책 사용
- ✓ 큐함수 업데이트에는 벨만 최적 방정식을 이용한다.
 - 다음큐 함수중에서 가장값이 큰 큐함수를 이용해서 현재 큐함수를 업데이트

```
Initialize  $Q(s, a)$  arbitrarily
Repeat (for each episode):
  Initialize  $s$ 
  Repeat (for each step of episode):
    Choose  $a$  from  $s$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
    Take action  $a$ , observe  $r, s'$ 
     $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ 
     $s \leftarrow s'$ ;
  until  $s$  is terminal
```



몬테카를로 예측

$$V(s) \leftarrow V(s) + \alpha(G(s) - V(s))$$

시간차 예측

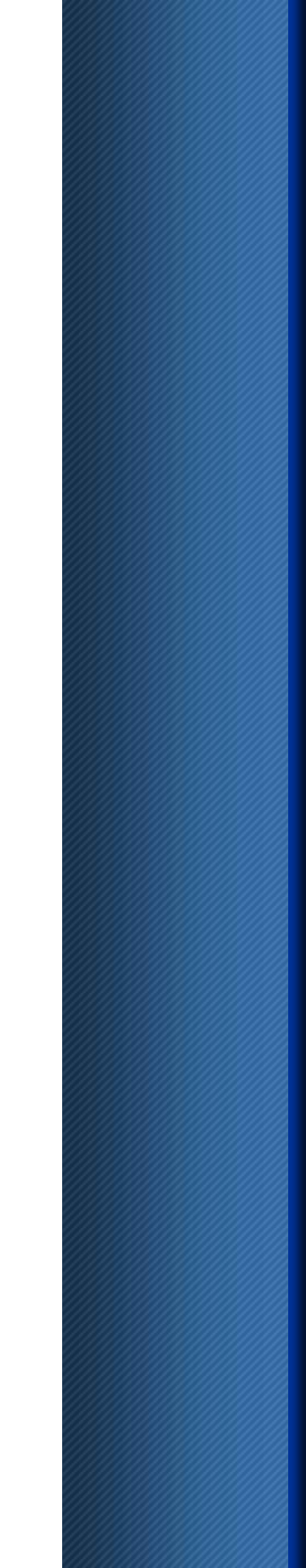
$$V(s_t) \leftarrow V(s_t) + \alpha(r_t + \gamma V(s_{t+1}) - V(s_t))$$

살사 E-greedy

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

큐러닝 E-greedy

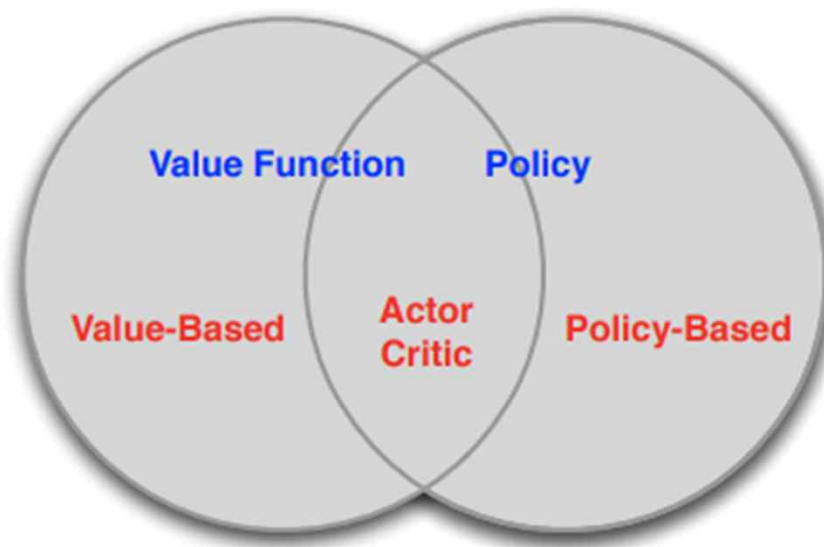
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t))$$

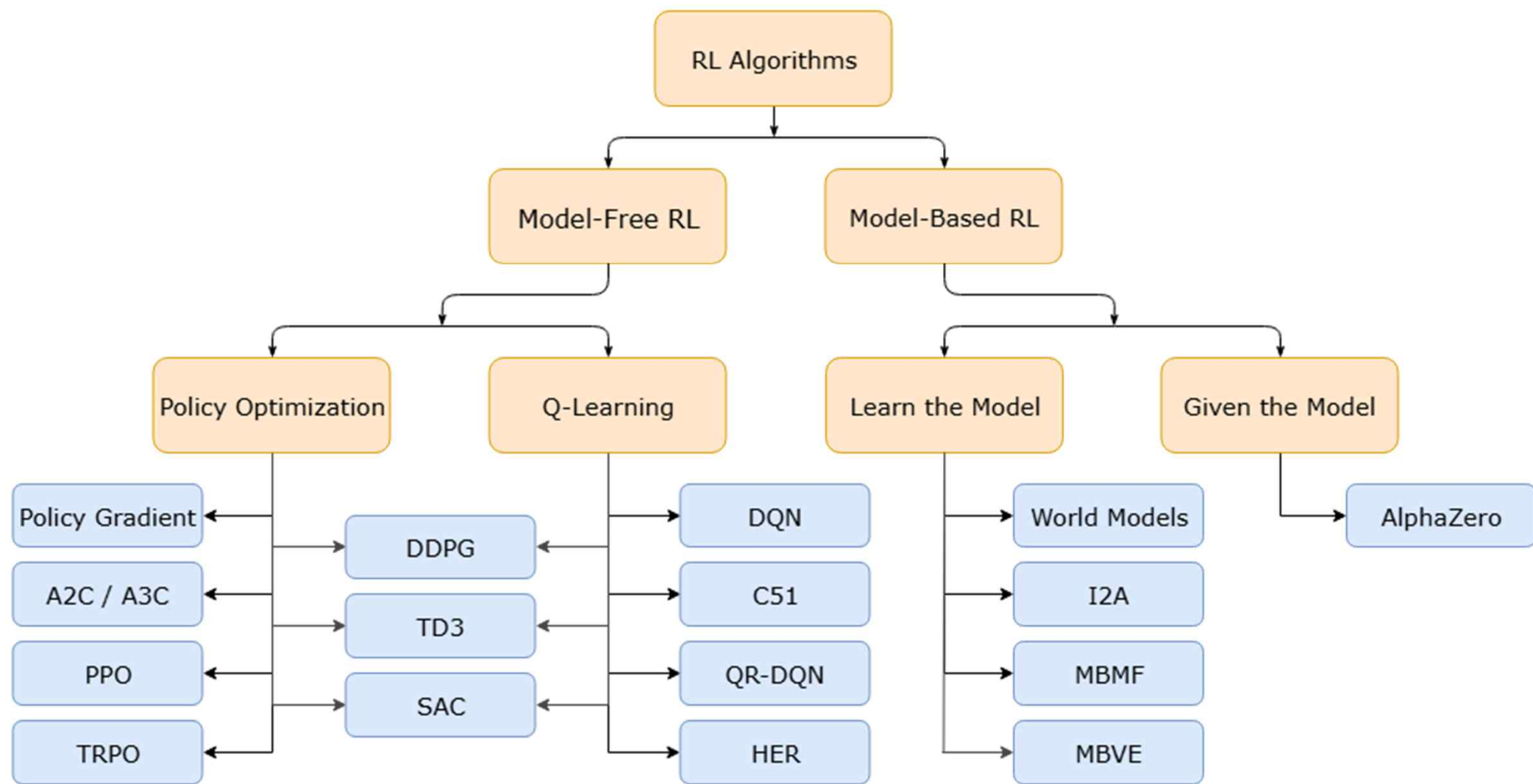


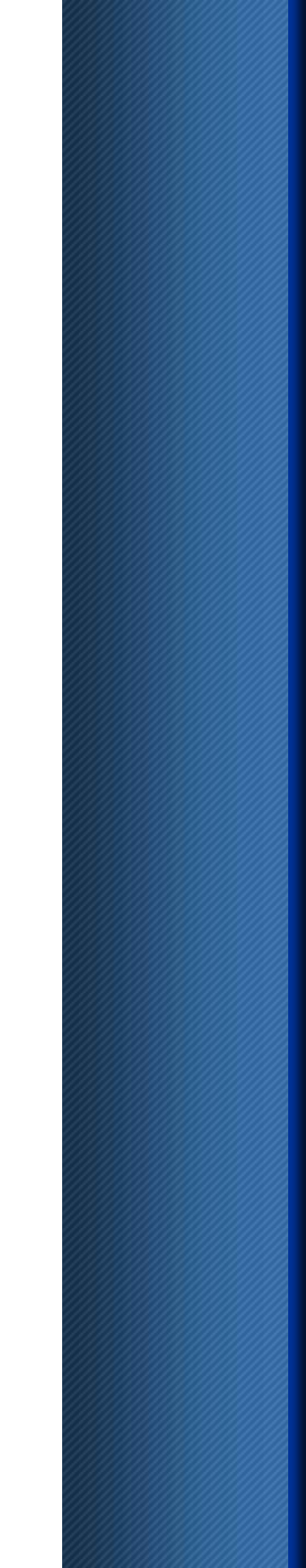
DeepRL

Value-Based and Policy-Based RL

- Value Based
 - Learnt Value Function
 - Implicit policy (e.g. ϵ -greedy)
- Policy Based
 - No Value Function
 - Learnt Policy
- Actor-Critic
 - Learnt Value Function
 - Learnt Policy







DQN

Generalization with deep learning

- ✓ Value function approximation with deep learning
- ✓ Large scale or infinite dimensional state can be solvable

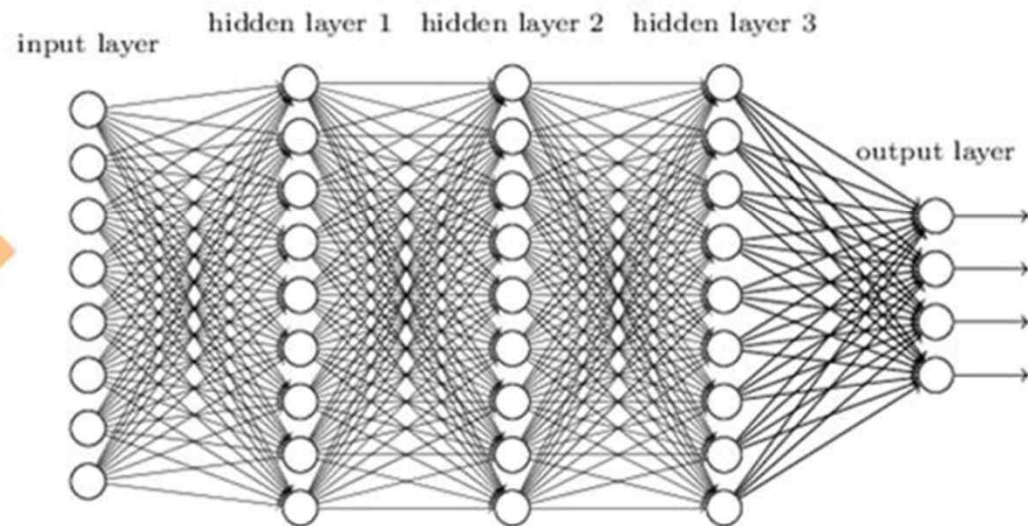
Tabular Q value function

$\hat{Q} =$

state \ action	A	B	C	D	E	F
A	-	-	-	-	80	-
B	-	-	-	64	-	100
C	-	-	-	64	-	-
D	-	80	51	-	80	-
E	64	-	-	64	-	100
F	-	80	-	-	80	100



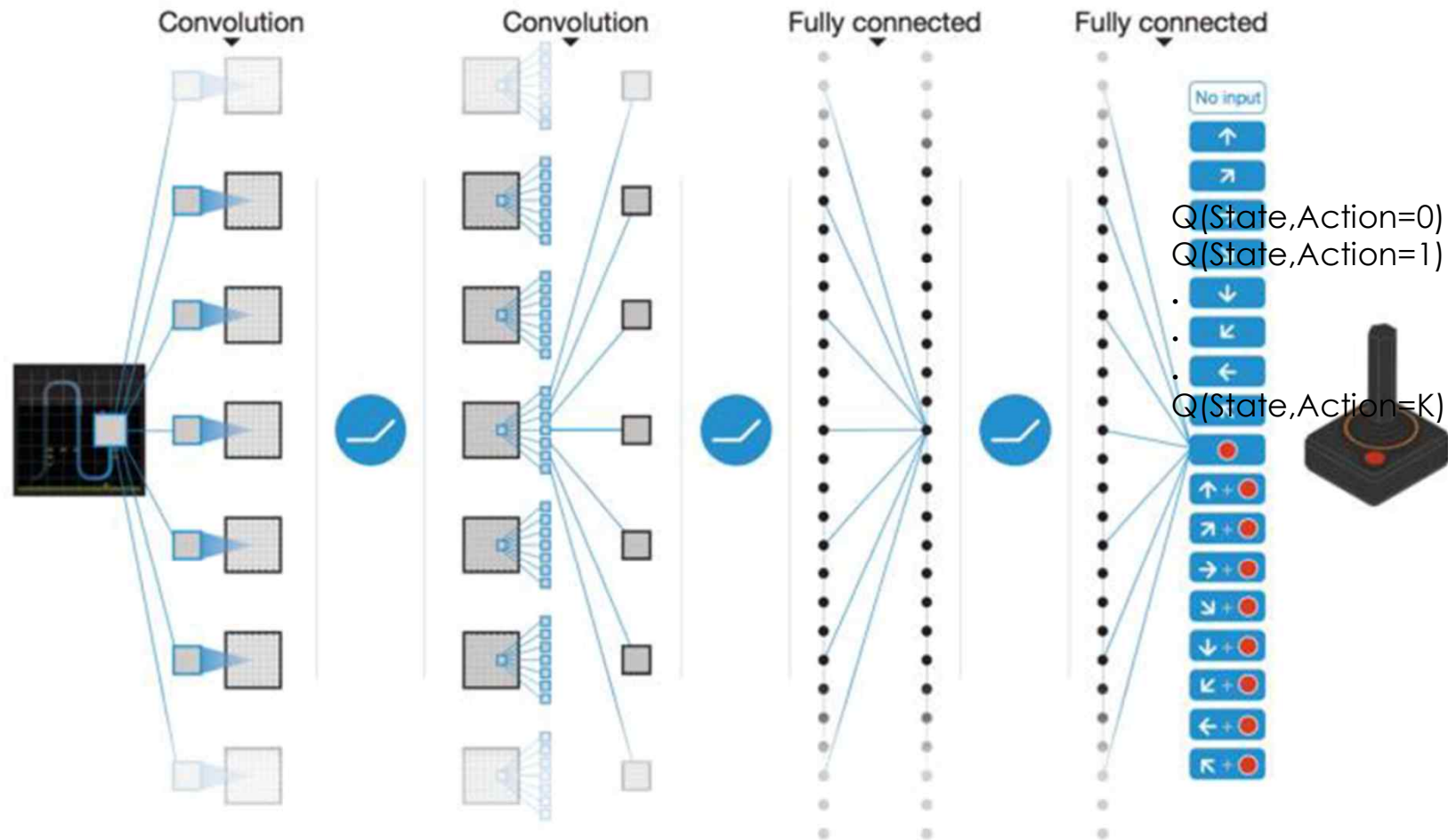
Deep neural network



- ✓ This needs supervised learning techniques and online moving target regression.

Deep Q Networks (DQN)

- ✓ Q learning에서 특정 state에 대한 action-value function으로 CNN을 사용



Loss function

- ✓ Q learning의 업데이트 식

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \overset{\text{TD error}}{\left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]}$$

- ✓ TD error의 크기를 Loss로 사용, regression 방식으로 학습

$$L = \frac{1}{2} \left[\underbrace{r + \max_{a'} Q(s', a')}_{\text{target}} - \underbrace{Q(s, a)}_{\text{prediction}} \right]^2$$

Target Network

$$L = \frac{1}{2} \left[\underbrace{r + \max_{a'} Q(s', a')}_{\text{target}} - \underbrace{Q(s, a)}_{\text{prediction}} \right]^2$$

- ✓ Moving Target Regression : Q를 학습하는 순간, target 값도 같이 변화 학습 성능 저하
- ✓ Target Network : DQN과 동일한 구조를 가지고 있으며 학습 도중 weight값이 변하지 않는 별도의 네트워크로 $Q(s', a')$ 로 고정된 target value를 사용하여 학습 안정성 개선
- ✓ (Target Network의 weight값들은 주기적으로 DQN의 것을 복사)

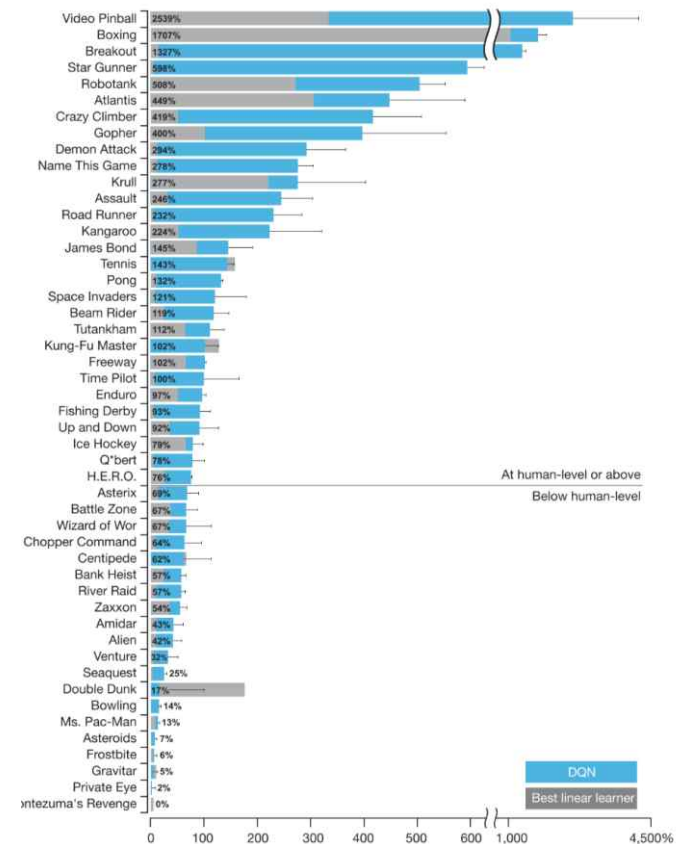
- ✓ **Data inefficiency** : 그 동안 경험한 experience들을 off-policy learning으로 재활용(Mini-batch learning)
- ✓ **Data correlation** : Mini-batch를 On-line으로 구성할 경우 Mini batch 내의 데이터들이 서로 비슷함(Correlated)
 - 한쪽으로 치우친 불균형한 학습이 발생함
- ✓ **Replay Memory** : (State, Action, Reward, Next State) 데이터를 Buffer에 저장해놓고 그 안에서 Random Sampling하여 Mini-batch를 구성하는 방법으로 해결

Reward Clipping

- ✓ **Reward Scale problem** : 도메인에 따라 Reward의 scale이 다르기 때문에 학습되는 Q-value의 크기도 제각각. 이렇게 학습해야하는 Q-value의 variance가 매우 큰 경우 (ex : -1000~1000) 신경망 학습이 어려움.
- ✓ **Reward Clipping**: Reward의 크기를 -1~1 사이로 잘라내어 안정적으로 학습

DQN의 성능

- ✓ •ATARI 2600 고전게임에서 실험
- ✓ •절반 이상의 게임에서 사람보다 우수
- ✓ •기존방식 (linear)에 비해 월등한 향상
- ✓ •일부 게임은 학습에 실패함.
- ✓ (sparse reward, 복잡한 단계를 가진 게임)



DQN의 발전된 모델들 - Double DQN

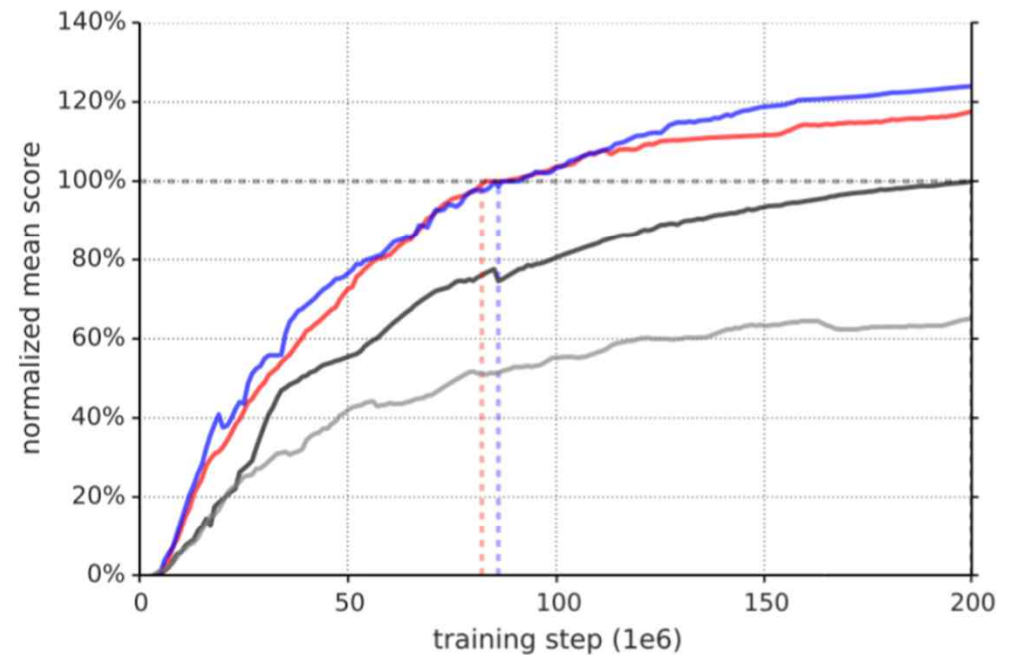
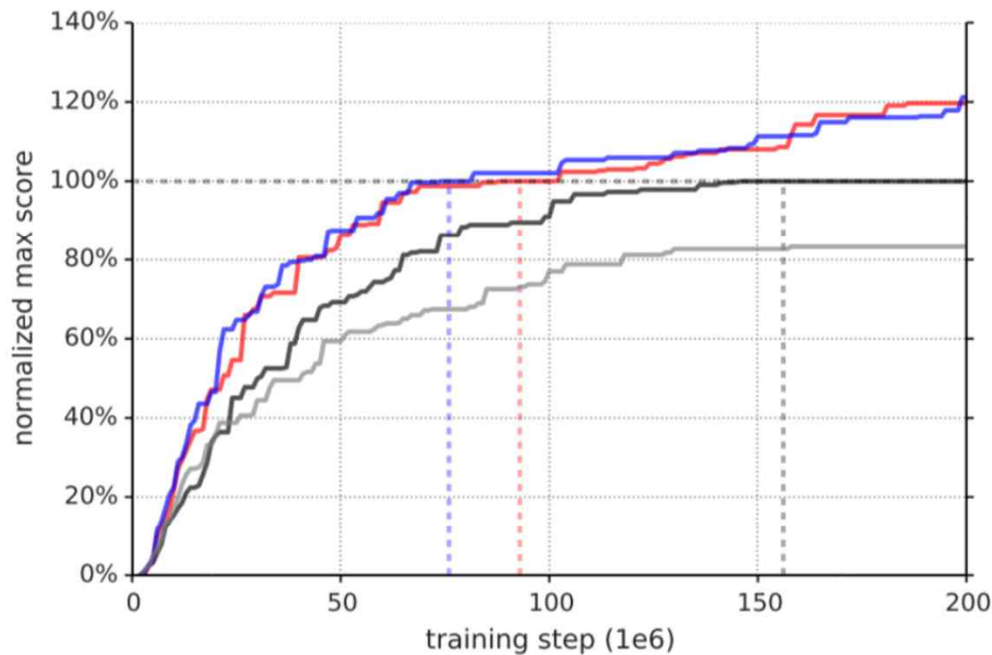
$$Y_t^{\text{DQN}} \equiv R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t^-).$$

$$Y_t^{\text{DoubleQ}} \equiv R_{t+1} + \gamma Q(S_{t+1}, \underset{a}{\operatorname{argmax}} Q(S_{t+1}, a; \theta_t); \theta'_t).$$

- ✓ Positive bias: max 연산자에 의해 Q-value를 실제보다 높게 평가하는 경향이 있음
- ✓ Double Q learning을 사용하여 이 현상을 방지, DQN에 보다 빠른 학습이 가능

DQN의 발전된 모델 - Prioritized Replay Memory

- ✓ 일반적인 DQN은 Replay Memory에서 Uniform Sampling
- ✓ TD error가 높은 데이터들이 선택될 확률을 높여, 더 빠르고 효율적인 학습이 가능



— uniform — rank-based — proportional — uniform DQN

Summary of Policy Gradient Algorithms

- The **policy gradient** has many equivalent forms

$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) \mathbf{v}_t]$	REINFORCE
$= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) Q^w(s, a)]$	Q Actor-Critic
$= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) A^w(s, a)]$	Advantage Actor-Critic
$= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) \delta]$	TD Actor-Critic
$= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) \delta \mathbf{e}]$	TD(λ) Actor-Critic
$G_{\theta}^{-1} \nabla_{\theta} J(\theta) = w$	Natural Actor-Critic

- Each leads a stochastic gradient ascent algorithm
- Critic uses **policy evaluation** (e.g. MC or TD learning) to estimate $Q^{\pi}(s, a)$, $A^{\pi}(s, a)$ or $V^{\pi}(s)$

DQN이 이론 점과 부족한 점

✓ 1. DQN01 이론 점

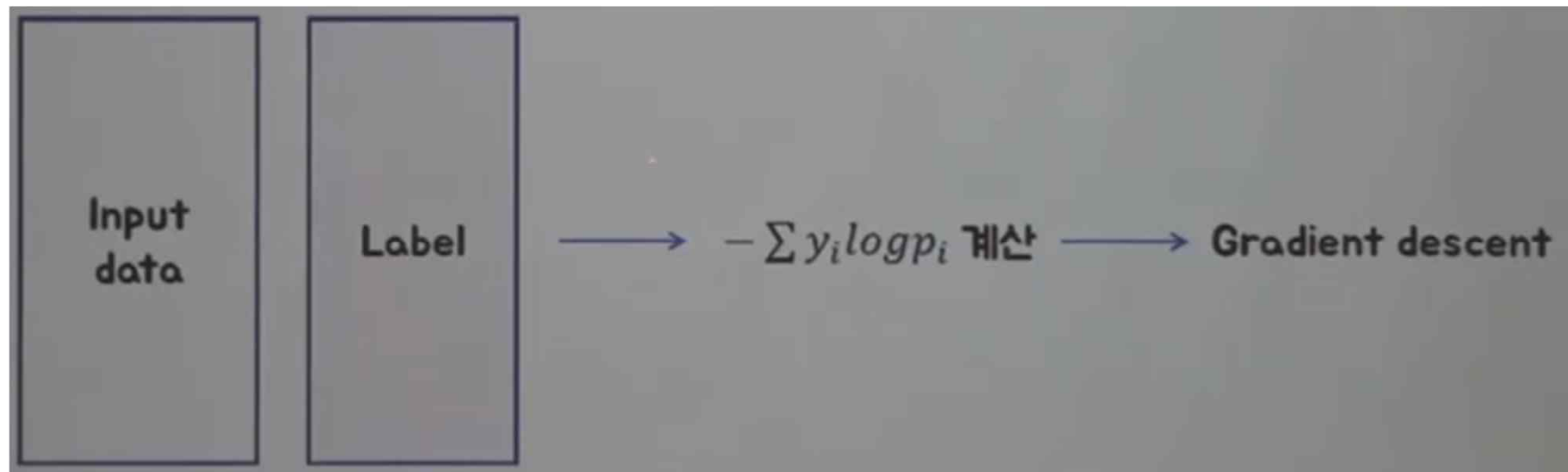
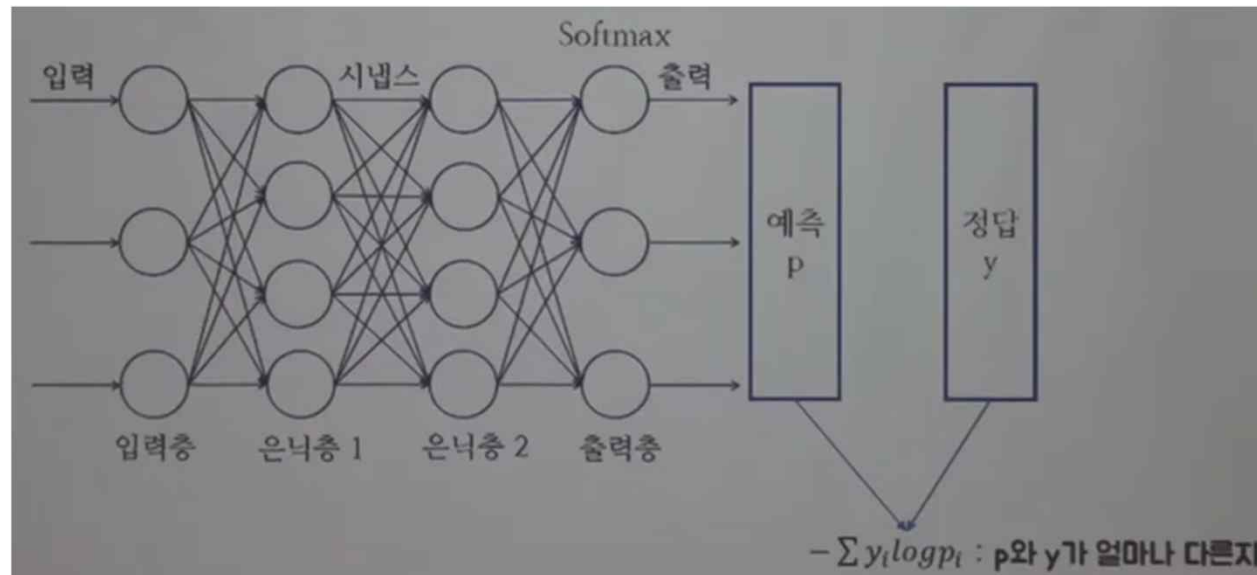
- 게임 화면을 상태 입력으로 받아서 학습(CNN)
- 사람보다 게임 플레이를 더 잘하는 에이전트
- 샘플들 사이의 강한 상관관계를 리플레이 메모리로 해결

✓ 2. DQN01 부족한 점

- 많은 메모리의 사용
- 느린 학습 속도
- 불안정한 학습 과정(가치함수에 대한 greedy policy이므로)

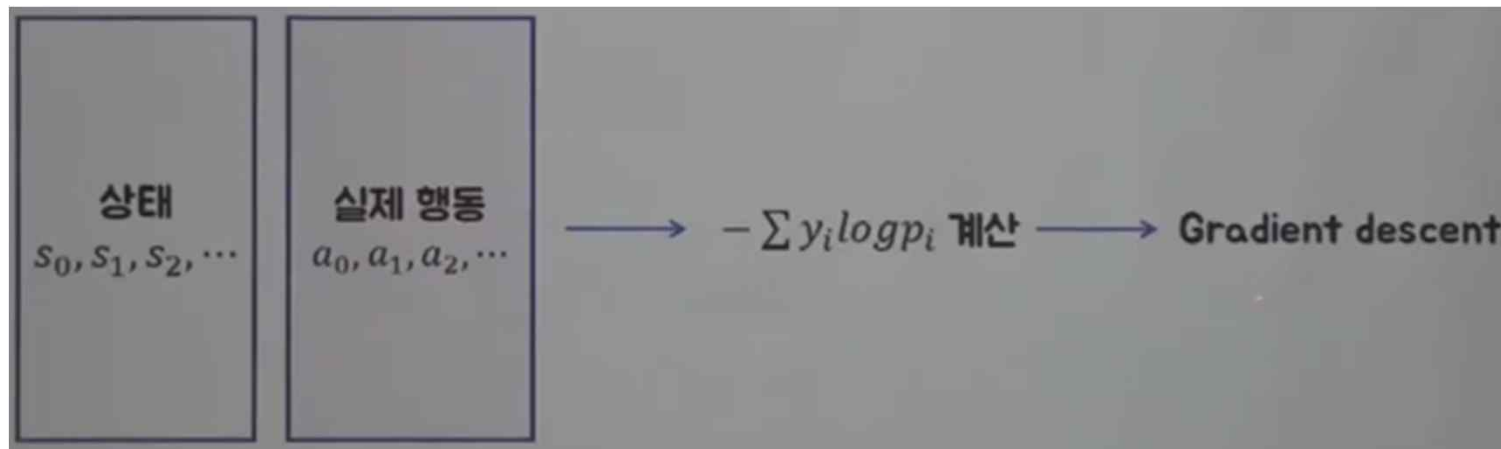
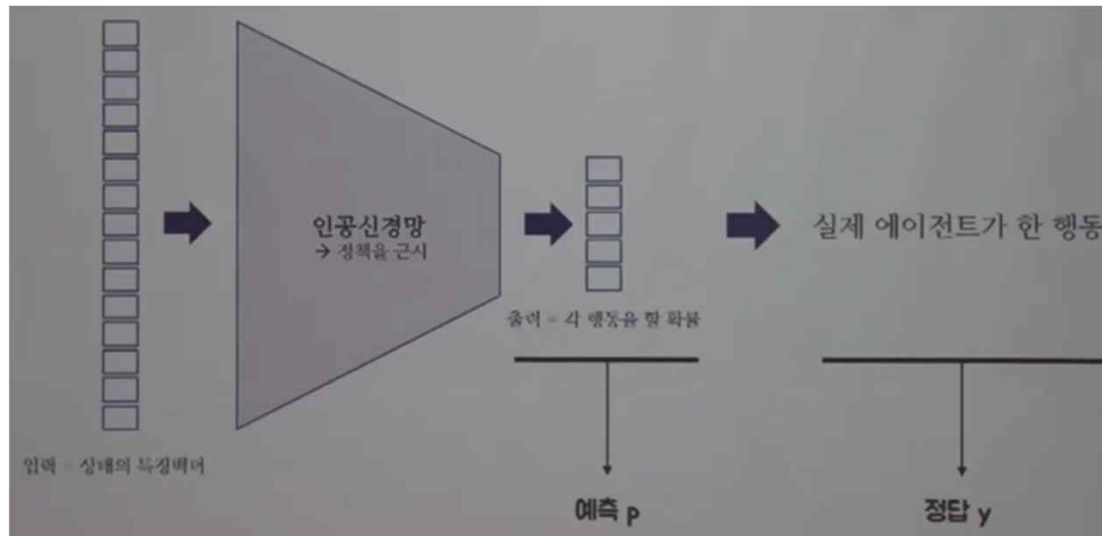
지도학습 Cross entropy

✓ 지도학습의 크로스 엔트로피(Cross entropy)



강화학습 Cross entropy

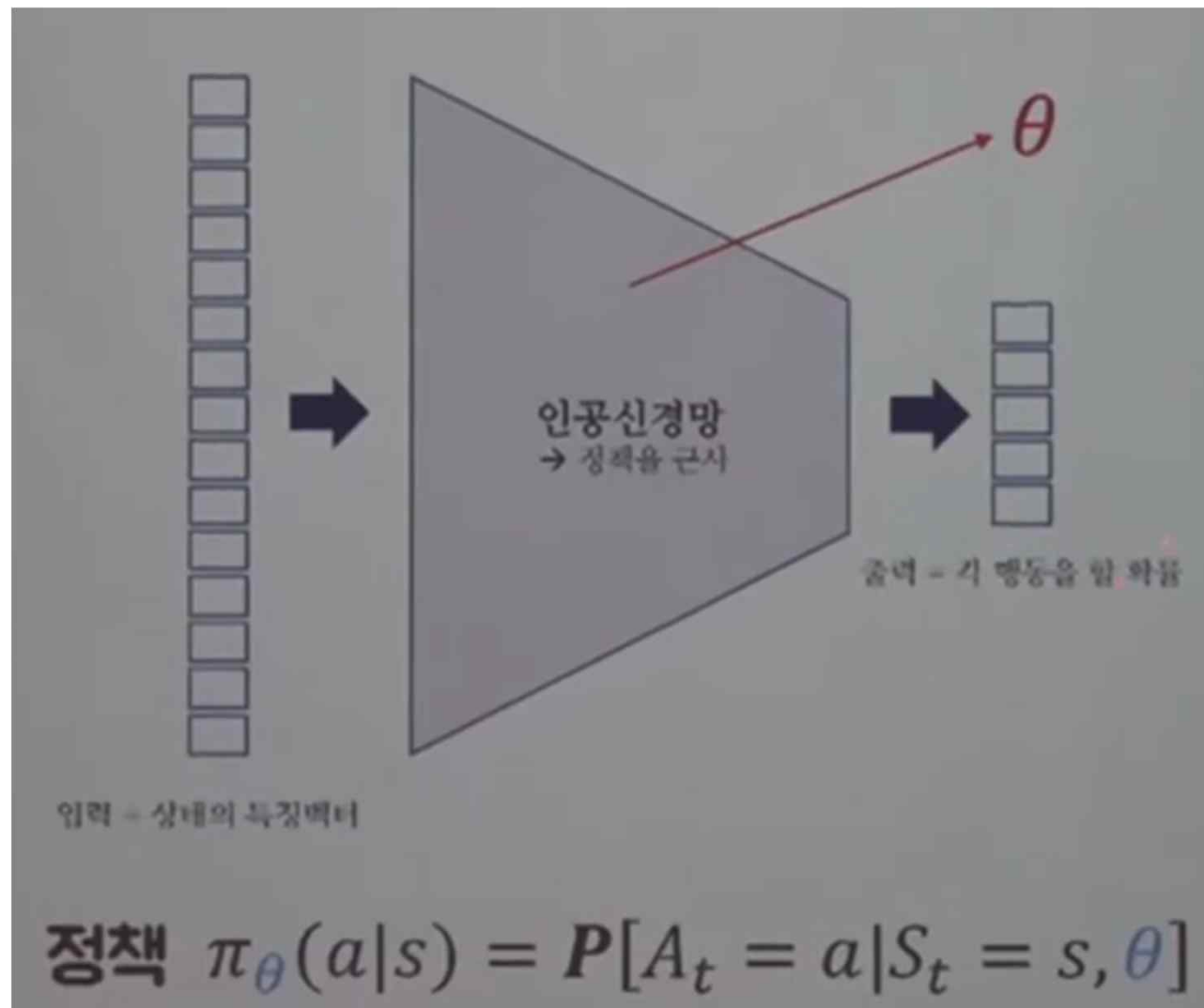
✓ 강화학습의 크로스 엔트로피(Cross entropy)



나의 예측을 실제 행동에 가깝게 만든다 → 어떤 의미 ??
업데이트의 방향성이 중요

정책의 근사화

- ✓ 인공지능망으로 정책을 근사 → 정책 신경망



목표함수 요약

1. 경로(Trajectory)의 개념

- 정책 π_θ 에 따라 에이전트가 환경과 상호작용하며 생성되는 상태·행동·보상의 기록
- 경로 예시

$$\tau = s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T$$

2. 목표함수 $J(\theta)$

- 에이전트가 정책을 따를 때 기대되는 총 보상
- 즉, "이 정책이 얼마나 좋은가?"를 수치로 표현한 함수

$$J(\theta) = \mathbb{E} \left[\sum_{t=0}^{T-1} r_{t+1} \mid \pi_\theta \right] = \mathbb{E}[r_1 + r_2 + r_3 + \dots + r_T \mid \pi_\theta]$$

3. 핵심 요약

- 정책이 바뀌면 경로가 바뀌고, 경로가 바뀌면 보상도 달라진다.
- 강화학습의 목표는
정책 파라미터 θ 를 조정하여 $J(\theta)$ 를 최대화하는 것.

Gradient ascent

Gradient Ascent에 의한 정책 업데이트

1. 목표

- 목표 함수 $J(\theta)$ 를 최대화하기 위해 정책 파라미터 θ 를 업데이트한다.

2. 핵심 아이디어

- $J(\theta)$ 가 증가하는 방향(경사 방향)으로 θ 를 조정한다.
- 즉, 경사 상승(Gradient Ascent) 방식으로 정책을 개선한다.

3. 업데이트 공식

$$\theta' = \theta + \alpha \nabla_{\theta} J(\theta)$$

$$\nabla_{\theta} J(\theta) = \text{Policy gradient}$$

- α : 학습률
- $\nabla_{\theta} J(\theta)$: Policy Gradient

4. 핵심 요약

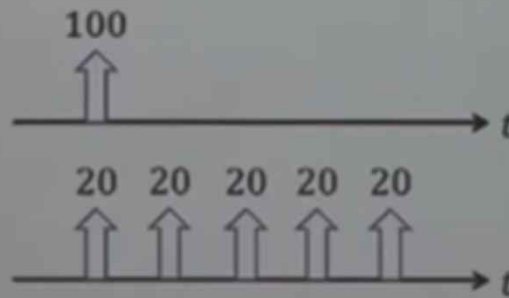
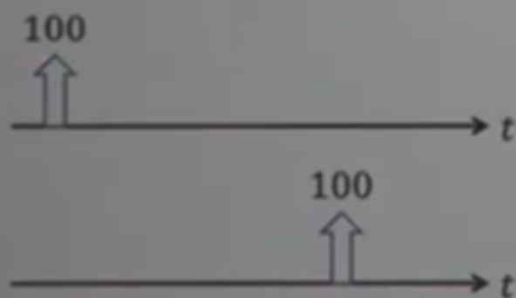
- Policy Gradient는
“보상이 높은 방향으로 정책을 개선하는 변화량”
을 의미한다.

REINFORCE

REINFORCE = 몬테카를로 Policy gradient

$$\nabla_{\theta} J(\theta) = E_{\tau} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t'=t+1}^T r_{t'} \right]$$

$\sum_{t'=t}^{T-1} r_{t'}$ 와 같은 단순 보상의 합 $\rightarrow$ 3가지 문제



$$0.1 + 0.1 + \dots = \infty$$
$$1 + 1 + \dots = \infty$$

감가율(discount factor) $0 \leq \gamma \leq 1$ 의 도입

REINFORCE

1. 한 에피소드를 현재 정책에 따라 실행
2. Trajectory를 기록
3. 에피소드가 끝난 뒤 G_t 계산
4. Policy gradient를 계산해서 정책 업데이트 $\text{Policy gradient} = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$
5. (1~4) 반복

에피소드 마다 업데이트 → 몬테카를로 Policy gradient = REINFORCE

REINFORCE의 문제

1. Variance가 높다
2. 에피소드마다 업데이트가 가능하다(on-line X)

몬테카를로 → TD(Temporal-Difference)

REINFORCE → Actor-Critic

Actor-Critic

Actor-Critic = REINFORCE + 실시간 학습

$$\nabla_{\theta} J(\theta) = E_{\tau} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q_{\pi_{\theta}}(s_t, a_t) \right]$$
$$\sim \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q_{\pi_{\theta}}(s_t, a_t)$$

만약 $Q_{\pi_{\theta}}(s_t, a_t)$ 를 알 수 있다면 매 time-step마다 업데이트하는 것이 가능!!

$Q_{\pi_{\theta}}(s_t, a_t) \sim Q_w(s_t, a_t)$ Let's make Critic!!!!

$$\nabla_{\theta} J(\theta) \sim \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q_w(s_t, a_t)$$

Advantage 함수 = 큐함수 - 베이스라인 $\rightarrow$ Variance를 낮춰준다

큐함수 : 특정 상태, 특정 행동에 따른 값

가치함수 : 특정 상태, 전반적 행동에 따른 값 $\rightarrow$ 베이스라인

$$\nabla_{\theta} J(\theta) \sim \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (Q_w(s_t, a_t) - V_v(s_t))$$

Actor-Critic

$$\nabla_{\theta} J(\theta) \sim \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (Q_w(s_t, a_t) - V_v(s_t))$$

Q와 V를 둘 다 근사하는 건(ex 뉴럴넷) 비효율적

$Q(s_t, a_t) = E[r_{t+1} + \gamma V(s_{t+1}) | s_t, a_t]$ 을 이용!

$$\nabla_{\theta} J(\theta) \sim \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (\underbrace{r_{t+1} + \gamma V_v(s_{t+1})}_{\text{큐함수}} - \underbrace{V_v(s_t)}_{\text{베이스라인}})$$

큐함수

베이스라인

1. Actor

1) 정책을 근사: θ

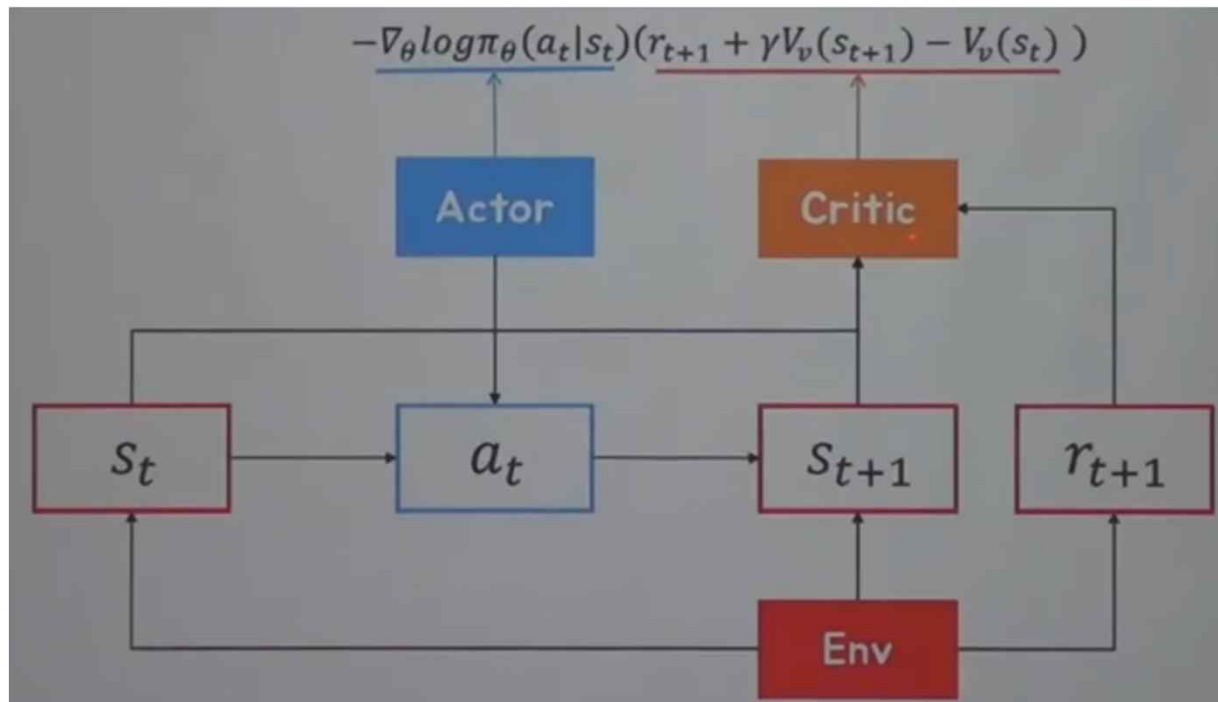
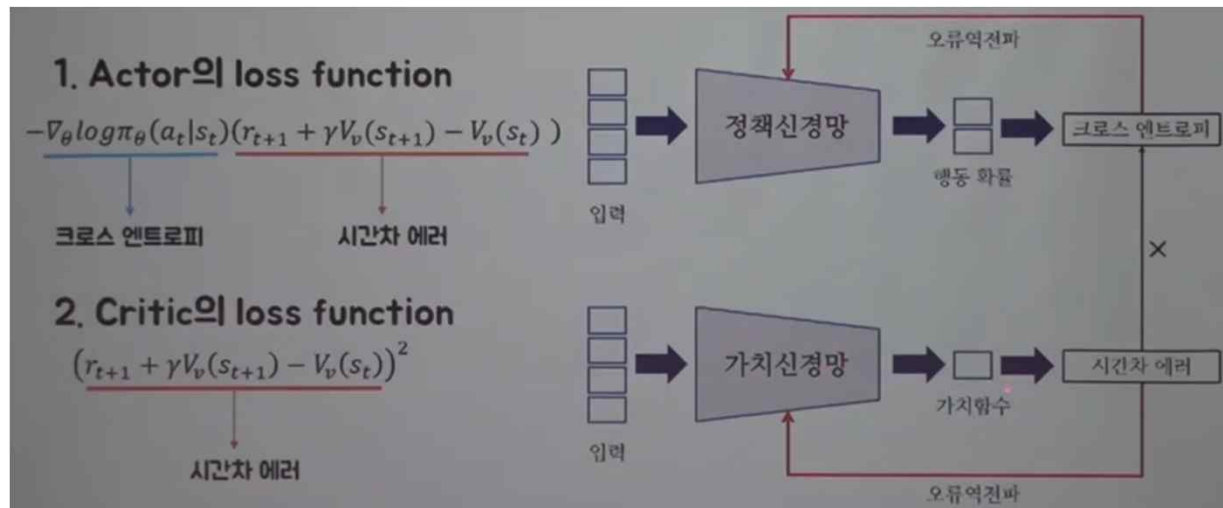
2) $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (r_{t+1} + \gamma V_v(s_{t+1}) - V_v(s_t))$ 로 업데이트

2. Critic

1) 가치함수(Value function)을 근사: v

2) $(r_{t+1} + \gamma V_v(s_{t+1}) - V_v(s_t))^2$ 의 오차함수로 업데이트

Actor-Critic



Idea

TRPO (Trust Region Policy Optimization)

- 기존 문제: “Policy를 너무 멀리 업데이트하면 성능이 내려간다”
- Trust-region = 신뢰할 수 있는 작은 영역 안에서만 업데이트.

PPO

- TRPO의 안정성 유지. 구현을 단순화함.

Main components

Loss

1. Clipped surrogate objective
2. Adaptive KL penalty

- **Constrained Form**

알고 있는 확률 분포로 변경

$$\begin{aligned} & \underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \\ & \text{subject to} \quad \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] \leq \delta. \end{aligned}$$

- **Penalized Form**

$$\begin{aligned} & \underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right] \\ & \quad \quad \quad \downarrow \\ & \quad \quad \quad r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad \text{so } r(\theta_{\text{old}}) = 1. \end{aligned}$$

Probability ratio

새 정책이 옛 정책 대비 얼마나 행동 확률을 늘리거나 줄였는가

중요도 샘플링 (Importance Sampling)

$$\begin{aligned} \mathbb{E}_{X \sim P}[f(X)] &= \sum P(X) f(X) \\ &= \sum Q(X) \frac{P(X)}{Q(X)} f(X) \\ &= \mathbb{E}_{X \sim Q} \left[\frac{P(X)}{Q(X)} f(X) \right] \end{aligned}$$

$$U(\theta) = \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\frac{P(\tau | \theta)}{P(\tau | \theta_{\text{old}})} R(\tau) \right]$$

$$\nabla_{\theta} U(\theta) = \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\frac{\nabla_{\theta} P(\tau | \theta)}{P(\tau | \theta_{\text{old}})} R(\tau) \right]$$

$$\nabla_{\theta} U(\theta)|_{\theta=\theta_{\text{old}}} = \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\frac{\nabla_{\theta} P(\tau | \theta)|_{\theta_{\text{old}}}}{P(\tau | \theta_{\text{old}})} R(\tau) \right]$$

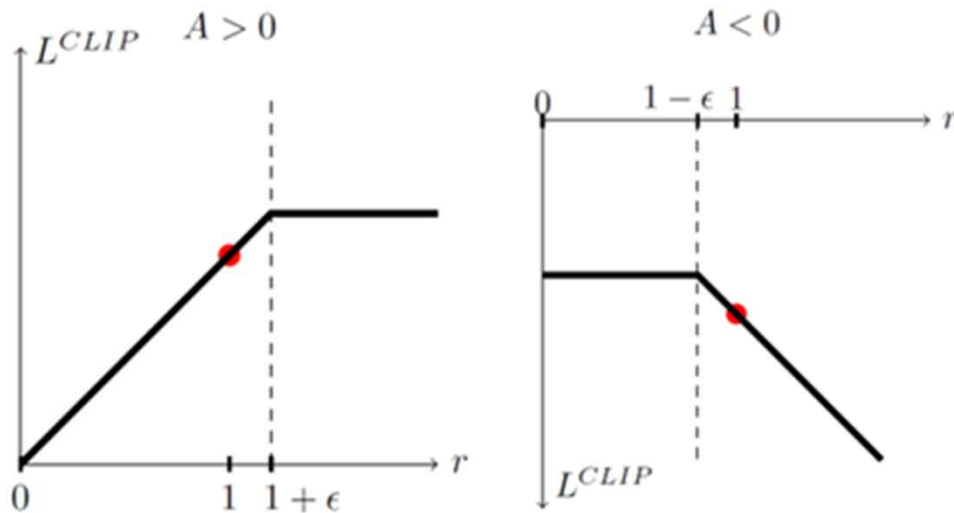
$$= \mathbb{E}_{\tau \sim \theta_{\text{old}}} [\nabla_{\theta} \log P(\tau | \theta)|_{\theta_{\text{old}}} R(\tau)]$$

Clipped Surrogate Objective

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\underbrace{\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)}_{\text{Original Loss}} \right]$$

Clipped Loss

Lower bound of unclipped objective = L^{CPI}



- θ_{old} 근처에서는 (즉, $r=1$ 근처) L^{CPI} 와 L^{CLIP} 가 1차 근사.

SAC(Soft Actor Critic)

1. SAC는 어떤 문제를 해결하려고 등장했는가?

기존 RL 알고리즘의 문제점

1. 데이터 효율이 낮음
 - PPO/A3C 등 on-policy 알고리즘은 한 번 사용한 데이터를 다시 쓰지 못함.
 - 로봇 제어처럼 샘플 비용이 큰 환경에서 비효율적임.
2. 탐색 부족 / 조기 수렴 문제
 - deterministic policy 기반(DDPG 등)은 탐색이 잘 안 됨.
3. 학습 불안정성
 - Q-function 과 Policy가 상호작용하면서 쉽게 발산하거나 흔들림.

2. SAC의 핵심 철학: “최대 엔트로피 강화학습(Maximum Entropy RL)”

SAC는 기존 목표 함수에 행동의 엔트로피(entropy)를 추가하여

“좋은 보상 + 행동 다양성”을 동시에 최대화합니다.

목표

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_t r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right]$$

즉,

- 보상은 크게
- 정책은 “너무 확신하지 말고”, 행동을 다양하게

정해진 범위 안에서 무작위성(탐색)을 유지하는 정책을 학습합니다.

이게 SAC의 본질입니다.

3. SAC 구조 요약: Actor + (Twin) Critic + Temperature α

SAC는 네 가지 구성요소가 있습니다.

1) 정책(Actor) $\pi(a|s)$

- Gaussian(정규분포) 정책 사용
- 행동 샘플링:

$$a \leftarrow \tanh(\mu_{\theta}(s) + \sigma_{\theta}(s) \cdot \epsilon)$$

2) Q-function (Twin Critic)

- Q_1, Q_2 두 개를 학습합니다.
- 둘 중 작은 값 사용
(overestimation 문제 해결; TD3의 아이디어 재사용)

$$Q_{\min}(s, a) = \min(Q_1(s, a), Q_2(s, a))$$

3) Target Critic

- Q-function의 안정성을 위해 Target Network 사용
- Soft Update

$$\theta_{\text{target}} \leftarrow \tau \theta + (1 - \tau) \theta_{\text{target}}$$

4) 엔트로피 온도 파라미터 α

- α 가 높으면 탐색 증가
- 낮으면 exploit 증가
- 자동 조정(auto-tuning)이 가능 (SAC의 특징)

SAC(Soft Actor Critic)

4. SAC 업데이트 단계 (핵심 흐름)

SAC는 매 스텝 다음 과정으로 진행됩니다.

Step 1. Critic(Q-function) 업데이트

SAC의 목표는 Bellman backup을 수행하는 것입니다.

Target Q 계산

$$a' \sim \pi(\cdot|s')$$
$$y = r + \gamma (Q_{\min}(s', a') - \alpha \log \pi(a'|s'))$$

여기서

$$-\alpha \log \pi(a'|s')$$

는 엔트로피 항입니다.

Critic Loss

$$L_Q = \mathbb{E} [(Q_1(s, a) - y)^2 + (Q_2(s, a) - y)^2]$$

Critic를 먼저 업데이트합니다.

Step 2. Actor 업데이트 (Policy Improvement)

목표:

π 가 Q 를 가장 잘 높이는 방향으로 학습 + 엔트로피 증가

Actor Loss:

$$L_\pi = \mathbb{E}_{s \sim \mathcal{D}} [\alpha \log \pi(a|s) - Q_{\min}(s, a)]$$

즉,

- Q 가 높은 행동을 선택하도록(good action)
- $\log \pi$ 항을 크게 만들어 정책이 덜 확신하도록(entropy 유지)

하는 목적 함수.

Step 3. Temperature α 업데이트 (Auto-tuning)

목표 엔트로피 $\mathcal{H}_0$ 를 정해 놓고

실제 엔트로피가 이 목표를 유지하도록 α 를 자동으로 조절합니다.

$$L_\alpha = \mathbb{E}_{a \sim \pi} [-\alpha (\log \pi(a|s) + \mathcal{H}_0)]$$

- 엔트로피가 목표보다 낮으면 $\rightarrow \alpha$ 증가 $\rightarrow$ 더 랜덤해짐
- 너무 높으면 $\rightarrow \alpha$ 감소 $\rightarrow$ 더 결정적으로

이 자동 온도 조절이 SAC의 강점입니다.

Step 4. Target Critic soft update

$$\theta_{\text{target}} \leftarrow \tau \theta + (1-\tau) \theta_{\text{target}}$$

TD 타깃 값을 부드럽게 업데이트하여 안정성을 높입니다.

SAC(Soft Actor Critic)

5. SAC의 전체 Loop (요약)

1. 경험 수집 (Replay Buffer)
2. Q-function 두 개 업데이트
3. Actor 업데이트
4. Temperature α 업데이트
5. Target network soft update
6. 반복

6. 왜 SAC가 로봇 제어에 많이 사용되는가?

1. 샘플 효율이 매우 좋음 (off-policy)

Replay Buffer로 데이터 재사용.

2. 안정적이고 튼튼한 학습

- Twin Critic
- Target network
- Soft update
- Maximum entropy → 덜 overfit됨

3. 실제 로봇에서는 탐색이 필수적이지만 위험

- 엔트로피 기반 탐색 덕분에 deterministic DDPG보다 안정적
- α auto-tuning이 위험한 탐색을 자동 조절함

4. 행동의 연속 공간(Continuous Action)에 탁월

로봇 제어는 90% 이상 continuous action.

7. SAC 수식 요약

SAC의 최종 목적 함수

$$\max_{\pi} \mathbb{E} \left[\sum_t r_t + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right]$$

Critic Bellman Backup

$$y = r + \gamma(Q_{\min}(s', a') - \alpha \log \pi(a'|s'))$$

Actor Loss

$$L_{\pi} = \mathbb{E} [\alpha \log \pi(a|s) - Q_{\min}(s, a)]$$

8. SAC를 한 줄로 요약하면

“보상 + 엔트로피를 모두 최대화하는 정책을
Replay Buffer 기반으로 안정적이고 효율적으로 학습시키는 RL 알고리즘”

이게 SAC의 본질입니다.

SAC까지의 전체 진화 흐름 요약

모델	개선 내용	SAC에 미친 영향
REINFORCE	최초의 Policy Gradient	Actor 구조의 기본
Actor-Critic	Baseline으로 분산 감소	Actor + Critic 틀 확립
DDPG	Off-policy + 연속제어	Replay Buffer, Target Net
TRPO	안정적 정책 업데이트	안정성 개념
PPO	간단한 stable 업데이트	정책 업데이트 안정성
TD3	Over-estimation 해결	Twin Q, smoothing 기법
SAC	Max Entropy, α 자동 조절, Off-policy 최강	최종 통합형 모델

- ✓ 결론적으로 SAC는 TD3의 안정성 + PPO/TRPO의 안정적 업데이트 철학 + DDPG의 Off-policy 효율 + 엔트로피 기반 탐색 을 모두 통합한 강력한 강화학습 모델입니다.



감사합니다.

e2g1234@naver.com
